

LAPORAN PRAKTIKUM STRUKTUR DATA

Double Linked List

DISUSUN OLEH:

Rayya Syaquinah Putri Hasibuan

2511532007

DOSEN PENGAMPU:

Dr. Wahyudi, S.T. M.T

ASISTEN PRATIKUM:

M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban pelaksanaan praktikum struktur data dengan judul Double Linked List. Laporan ini bertujuan untuk mendokumentasikan prosedur, hasil, serta pembahasan dari kegiatan praktikum sekaligus sebagai sarana pembelajaran bagi penulis untuk lebih memahami materi terkait.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu. Saran dan kritik yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu sehingga laporan ini dapat terselesaikan.

Padang, 11 Mei 2026

Rayya Syaquinah Putri Hasibuan

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Landasan Teori	3
2.2 Prosedur Praktikum.....	4
a) NodeDLL_2511532007	4
b) InsertDLL_2511532007	6
c) PenelusuranDLL_2511532007.....	9
d) HapusDLL_2511532007	11
BAB III KESIMPULAN DAN SARAN	14
3.1 Kesimpulan	14
3.2 Saran	14
DAFTAR PUSTAKA	14
TUGAS PRAKTIKUM PEKAN 6	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Double Linked List merupakan salah satu struktur data linear yang digunakan untuk menyimpan dan mengelola data secara dinamis. Berbeda dengan array yang memiliki ukuran tetap, Double Linked List memungkinkan proses penambahan, penghapusan, dan penelusuran data dilakukan dengan lebih fleksibel. Pada struktur ini, setiap node memiliki dua pointer, yaitu pointer menuju node berikutnya (next) dan pointer menuju node sebelumnya (prev). Dengan adanya dua arah penelusuran tersebut, proses pengolahan data menjadi lebih mudah dan efisien. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami cara kerja Double Linked List serta mampu mengimplementasikan operasi dasar seperti penambahan, penghapusan, dan penelusuran node menggunakan bahasa pemrograman Java.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk memahami konsep dasar Double Linked List beserta cara implementasinya dalam bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu membuat node, menambahkan data di berbagai posisi, menghapus data, serta melakukan penelusuran maju dan mundur pada Double Linked List. Dengan praktikum ini, mahasiswa diharapkan dapat memahami hubungan antar node melalui pointer next dan prev serta mampu menerapkan struktur data ini dalam penyelesaian masalah pemrograman.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah mahasiswa dapat memahami penggunaan struktur data Double Linked List secara lebih nyata melalui implementasi program. Mahasiswa juga dapat melatih

kemampuan logika pemrograman dalam mengelola hubungan antar node dan memahami proses manipulasi data secara dinamis. Selain itu, praktikum ini membantu meningkatkan pemahaman tentang operasi dasar struktur data seperti penambahan, penghapusan, dan penelusuran data sehingga dapat menjadi dasar dalam mempelajari struktur data yang lebih kompleks di masa mendatang.

BAB II PEMBAHASAN

2.1 Landasan Teori

Double Linked List atau sering juga disebut Doubly Linked List merupakan salah satu jenis struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada Double Linked List memiliki tiga bagian utama, yaitu bagian data, pointer next yang menunjuk ke node berikutnya, dan pointer prev yang menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dalam dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan. Struktur dasar Double Linked List dapat digambarkan sebagai berikut:

$$\text{NULL} \leftarrow \text{Node1} \leftrightarrow \text{Node2} \leftrightarrow \text{Node3} \rightarrow \text{NULL}$$

Berbeda dengan Single Linked List yang hanya memiliki satu pointer menuju node berikutnya, Double Linked List memungkinkan navigasi data menjadi lebih fleksibel karena setiap node saling terhubung dua arah. Oleh karena itu, struktur data ini banyak digunakan pada sistem yang membutuhkan perpindahan maju dan mundur secara mudah, seperti fitur undo dan redo pada aplikasi, navigasi back dan forward pada browser, serta sistem playlist lagu.

Pada Double Linked List non circular, node pertama disebut head dan node terakhir memiliki pointer next bernilai NULL, sedangkan pointer prev pada node pertama juga bernilai NULL. Untuk melakukan manipulasi data pada linked list, biasanya digunakan sebuah pointer kepala atau head yang selalu menunjuk ke node pertama. Setiap node baru yang dibuat awalnya memiliki pointer next dan prev bernilai NULL, kemudian pointer tersebut akan diperbarui sesuai posisi node di dalam linked list.

Double Linked List memiliki beberapa kelebihan dibandingkan struktur linked list biasa. Salah satu kelebihannya adalah dapat melakukan penelusuran dua arah sehingga akses data

menjadi lebih mudah dan cepat. Selain itu, proses penghapusan node juga lebih efisien karena node sebelumnya dapat langsung diketahui melalui pointer prev. Struktur ini juga memberikan fleksibilitas yang lebih tinggi dalam pengelolaan data dinamis.

Namun, Double Linked List juga memiliki beberapa kekurangan. Karena setiap node memiliki dua pointer, maka penggunaan memori menjadi lebih besar dibandingkan Single Linked List. Selain itu, proses implementasinya lebih kompleks karena setiap operasi harus memperhatikan dua hubungan pointer sekaligus, yaitu next dan prev.

Dalam Double Linked List terdapat beberapa operasi dasar yang sering digunakan, yaitu penyisipan node di awal, penyisipan node di akhir, penyisipan node pada posisi tertentu, penghapusan node di awal, penghapusan node di akhir, penghapusan node pada posisi tertentu, serta penelusuran maju dan mundur. Pada proses penyisipan di awal, node baru akan dijadikan head baru dan pointer akan diperbarui agar node lama tetap terhubung. Pada penyisipan di akhir, program harus menelusuri node hingga posisi terakhir sebelum menambahkan node baru. Sedangkan pada proses penghapusan, hubungan antar node harus diperbarui agar linked list tetap tersusun dengan benar setelah node tertentu dihapus.

Dengan memahami konsep Double Linked List, mahasiswa dapat mempelajari bagaimana data dapat dikelola secara dinamis menggunakan struktur yang saling terhubung. Pemahaman ini sangat penting sebagai dasar dalam mempelajari struktur data dan algoritma yang lebih kompleks pada pemrograman komputer.

2.2 Prosedur Praktikum

a) NodeDLL_2511532007

```

1 package pekan6_2511532007;
2
3 public class NodeDLL_2511532007 {
4     //mendefinisikan kelas node
5     int data_2007; //data
6     NodeDLL_2511532007 next_2007; //pointer ke next node
7     NodeDLL_2511532007 prev_2007; //pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511532007(int data_2007) {
11        this.data_2007=data_2007;
12        this.next_2007=null;
13        this.prev_2007=null;
14    }
15
16 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)
Program diawali dengan package pekan6_2511532007; untuk mengelompokkan file Java. Kemudian dibuat public class NodeDLL_2511532007 sebagai kelas utama yang digunakan untuk membentuk node pada Double Linked List.
2. Deklarasi variabel data
Variabel int data_2007; digunakan untuk menyimpan nilai atau isi data pada node.
3. Deklarasi pointer next
Variabel NodeDLL_2511532007 next_2007; digunakan untuk menyimpan alamat node berikutnya pada Double Linked List.
4. Deklarasi pointer prev
Variabel NodeDLL_2511532007 prev_2007; digunakan untuk menyimpan alamat node sebelumnya sehingga linked list dapat diakses dua arah.
5. Konstruktor NodeDLL_2511532007()
Konstruktor public NodeDLL_2511532007(int data_2007) digunakan untuk membuat node baru sekaligus mengisi nilai data pada node tersebut.
6. Mengisi data node
Baris this.data_2007 = data_2007; digunakan untuk memasukkan nilai data ke dalam node saat objek dibuat.
7. Inisialisasi pointer next
Baris this.next_2007 = null; digunakan untuk mengatur

pointer next menjadi null karena node baru belum terhubung dengan node berikutnya.

8. Inisialisasi pointer prev

Baris `this.prev_2007 = null;` digunakan untuk mengatur pointer prev menjadi null karena node baru belum memiliki hubungan dengan node sebelumnya.

9. Alur kerja program

Program dimulai dengan membuat sebuah node baru menggunakan konstruktor. Data yang diberikan akan disimpan ke dalam node, sedangkan pointer next dan prev diatur bernilai null. Node ini nantinya dapat dihubungkan dengan node lain untuk membentuk struktur Doubly Linked List yang saling terhubung dua arah.

Program ini menunjukkan dasar pembuatan node pada Double Linked List menggunakan Java. Setiap node memiliki data serta dua pointer, yaitu pointer ke node berikutnya dan sebelumnya. Dengan struktur ini, proses pengelolaan data menjadi lebih fleksibel karena data dapat ditelusuri dari depan maupun dari belakang.

b) InsertDLL_2511532007

```

1 package pekaad_2511532007;
2
3 public class InsertDLL_2511532007 {
4     //menambahkan node di awal dll
5     static NodeDLL_2511532007 insertBegin(NodeDLL_2511532007 head_2007, int data_2007) {
6         // buat node baru
7         NodeDLL_2511532007 new_node = new NodeDLL_2511532007(data_2007);
8         // jadikan pointer pertama head
9         new_node.next_2007 = head_2007;
10        // jadikan pointer dari head ke new node
11        if (head_2007 != null) {
12            head_2007.prev_2007 = new_node;
13        }
14        return new_node;
15    }
16    //fungsi menambahkan node di akhir
17    public static NodeDLL_2511532007 insertEnd(NodeDLL_2511532007 head_2007, int newData_2007) {
18        //buat node baru
19        NodeDLL_2511532007 newNode = new NodeDLL_2511532007(newData_2007);
20        //jika dll null jadikan head
21        if (head_2007 == null) {
22            head_2007 = newNode;
23        }
24        else {
25            NodeDLL_2511532007 curr_2007 = head_2007;
26            while (curr_2007.next_2007 != null) {
27                curr_2007 = curr_2007.next_2007;
28            }
29            curr_2007.next_2007 = newNode;
30            curr_2007.prev_2007 = curr_2007;
31        }
32        return head_2007;
33    }
34 }

```

```

16 // Fungsi menambahkan node di bagian akhir
17 public static NodeDLL_2511532007 insertAtFunction(NodeDLL_2511532007 head_2007, int pos_2007, int new_data_2007) {
18     //head node baru
19     NodeDLL_2511532007 new_node = new NodeDLL_2511532007(new_data_2007);
20     if (pos_2007 == 1) {
21         new_node.next_2007 = head_2007;
22         if (head_2007 != null) {
23             head_2007.prev_2007 = new_node;
24         }
25         head_2007 = new_node;
26         return head_2007;
27     }
28     NodeDLL_2511532007 curr_2007 = head_2007;
29     for (int i = 1; i < pos_2007 - 1 && curr_2007 != null; ++i) {
30         curr_2007 = curr_2007.next_2007;
31     }
32     if (curr_2007 == null) {
33         System.out.println("posisi tidak ada");
34         return head_2007;
35     }
36     new_node.prev_2007 = curr_2007;
37     new_node.next_2007 = curr_2007.next_2007;
38     curr_2007.next_2007 = new_node;
39     if (new_node.next_2007 != null) {
40         new_node.next_2007.prev_2007 = new_node;
41     }
42     return head_2007;
43 }
44
45 public static void printList(NodeDLL_2511532007 head_2007) {
46     NodeDLL_2511532007 curr_2007 = head_2007;
47     while (curr_2007 != null) {
48         System.out.print(curr_2007.data_2007 + " ");
49         curr_2007 = curr_2007.next_2007;
50     }
51     System.out.println();
52 }
53
54 public static void main(String[] args) {
55     //inisialisasi dll
56     NodeDLL_2511532007 head_2007 = new NodeDLL_2511532007(1);
57     head_2007.next_2007 = new NodeDLL_2511532007(2);
58     head_2007.next_2007.prev_2007 = head_2007;
59     head_2007.next_2007.next_2007 = new NodeDLL_2511532007(3);
60     head_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007;
61 }
62
63 // contoh dll
64 System.out.print("dll awal: ");
65 printList(head_2007);
66 // contoh di awal
67 head_2007 = insertBegin(head_2007, 1);
68 System.out.print("contoh 1 ditambah di awal: ");
69 printList(head_2007);
70 // contoh di akhir
71 System.out.print("contoh 2 ditambah di akhir: ");
72 int data_2007 = 4;
73 head_2007 = insertEnd(head_2007, data_2007);
74 printList(head_2007);
75 // contoh node di posisi
76 int data2 = 5;
77 int pos_2007 = 4;
78 head_2007 = insertAtFunction(head_2007, pos_2007, data2);
79 printList(head_2007);
80 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)

Program diawali dengan package pekan6_2511532007; untuk mengelompokkan file Java. Kemudian dibuat public class InsertDLL_2511532007 sebagai kelas utama yang berisi implementasi penambahan data pada Double Linked List.

2. Method insertBegin() (menambah node di awal)

Method ini digunakan untuk menambahkan node di bagian awal Double Linked List. Program membuat node baru, lalu menghubungkan node baru tersebut dengan head sebelumnya. Jika linked list tidak kosong, maka pointer prev dari head lama akan diarahkan ke node baru. Setelah itu node baru menjadi head yang baru.

3. Method insertEnd() (menambah node di akhir)

Method ini digunakan untuk menambahkan node di bagian akhir Double Linked List. Jika linked list masih kosong, maka node baru langsung dijadikan head. Jika sudah ada data, program akan menelusuri node hingga ke bagian

terakhir, kemudian menambahkan node baru di akhir linked list.

4. Method `insertAtPosition()` (menambah node pada posisi tertentu)

Method ini digunakan untuk menambahkan node pada posisi tertentu di dalam Double Linked List. Jika posisi yang dimasukkan adalah posisi pertama, maka node akan ditambahkan di awal. Jika posisi lainnya, program akan menelusuri linked list hingga posisi yang dituju, kemudian menyisipkan node baru di antara node sebelumnya dan node berikutnya. Jika posisi tidak ditemukan, program akan menampilkan pesan bahwa posisi tidak ada.

5. Method `printList()` (menampilkan isi linked list)

Method ini digunakan untuk menampilkan seluruh isi Double Linked List dari awal hingga akhir. Data ditampilkan secara berurutan dengan tanda `<->` sebagai penghubung antar node.

6. Method `main()` (program utama)

Method `main` digunakan untuk menjalankan program. Pertama dibuat Double Linked List awal berisi data 2 `<->` 3 `<->` 5. Setelah itu linked list ditampilkan. Selanjutnya dilakukan beberapa operasi penambahan data, yaitu menambah node 1 di awal, menambah node 6 di akhir, dan menambah node 4 pada posisi ke-4. Setiap perubahan langsung ditampilkan untuk melihat hasil linked list terbaru.

7. Alur kerja program

Program dimulai dengan membuat Double Linked List awal. Setelah itu program melakukan beberapa proses penambahan node, yaitu di awal, di akhir, dan pada posisi tertentu. Setelah setiap proses penambahan, isi linked list akan diperbarui dan ditampilkan kembali.

Program ini menunjukkan proses penambahan node pada Double Linked List di berbagai posisi, yaitu di awal, di akhir, dan di tengah. Dengan adanya pointer next dan prev, data dapat diakses dua arah sehingga pengelolaan data menjadi lebih fleksibel dan dinamis.

c) PenelusuranDLL_2511532007

```

1 package pekan6_2511532007;
2
3 public class PenelusuranDLL_2511532007 {
4     //fungsi penelusuran maju
5     static void forwardTraversal (NodeDll_2511532007 head) {
6         //inisialisasi penelusuran dari head
7         NodeDll_2511532007 curr = head;
8         //fungsi cetak data
9         while (curr != null) {
10             //print data
11             System.out.print(curr.data_2007 + " <-> ");
12             //pindah ke node berikutnya
13             curr = curr.next_2007;
14         }
15         //print akhir
16         System.out.println();
17     }
18     //fungsi penelusuran mundur
19     static void backwardTraversal (NodeDll_2511532007 tail) {
20         //inisialisasi penelusuran dari akhir
21         NodeDll_2511532007 curr = tail;
22         //fungsi cetak data
23         while (curr != null) {
24             //print data
25             System.out.print(curr.data_2007 + " <-> ");
26             //pindah ke node sebelumnya
27             curr = curr.prev_2007;
28         }
29         //print akhir
30         System.out.println();
31     }
32     public static void main(String[] args) {
33         //create dll
34         NodeDll_2511532007 head = new NodeDll_2511532007 (1);
35         NodeDll_2511532007 second = new NodeDll_2511532007 (2);
36         NodeDll_2511532007 third = new NodeDll_2511532007 (3);
37
38         head.next_2007 = second;
39         second.prev_2007 = head;
40         second.next_2007 = third;
41         third.prev_2007 = second;
42
43         System.out.println("Penelusuran maju: ");
44         forwardTraversal(head);
45
46         System.out.println("Penelusuran mundur: ");
47         backwardTraversal(third);
48     }
49 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)
Program diawali dengan package pekan6_2511532007; untuk mengelompokkan file Java. Kemudian dibuat public class PenelusuranDLL_2511532007 sebagai kelas utama yang berisi implementasi penelusuran pada Double Linked List.
2. Method forwardTraversal() (penelusuran maju)
Method forwardTraversal() digunakan untuk melakukan penelusuran data dari awal menuju akhir Double Linked List. Penelusuran dimulai dari node head, kemudian program akan menampilkan data satu per satu sambil berpindah ke node berikutnya menggunakan pointer next_2007 hingga mencapai node terakhir.

3. Method `backwardTraversal()` (penelusuran mundur)
Method `backwardTraversal()` digunakan untuk melakukan penelusuran data dari akhir menuju awal Double Linked List. Penelusuran dimulai dari node terakhir (tail), kemudian program akan menampilkan data sambil berpindah ke node sebelumnya menggunakan pointer `prev_2007` hingga mencapai node pertama.
4. Menampilkan data node
Pada kedua method traversal, program menggunakan:
`System.out.print(curr.data_2007 + " <-> ");`
Baris ini digunakan untuk menampilkan data setiap node secara berurutan dengan tanda penghubung `<->`.
5. Perpindahan node pada traversal maju
Baris: `curr = curr.next_2007;`
digunakan untuk memindahkan penelusuran ke node berikutnya saat traversal maju dilakukan.
6. Perpindahan node pada traversal mundur
Baris: `curr = curr.prev_2007;`
digunakan untuk memindahkan penelusuran ke node sebelumnya saat traversal mundur dilakukan.
7. Method `main()` (program utama)
Method `main` digunakan untuk menjalankan program. Pertama dibuat Double Linked List dengan data 1 <-> 2 <-> 3. Setelah node saling dihubungkan menggunakan pointer `next` dan `prev`, program melakukan penelusuran maju mulai dari node pertama, kemudian melakukan penelusuran mundur mulai dari node terakhir.
8. Alur kerja program
Program dimulai dengan membuat beberapa node dan menghubungkannya menjadi Double Linked List. Setelah itu program melakukan dua jenis penelusuran, yaitu penelusuran maju dari awal ke akhir dan penelusuran

mundur dari akhir ke awal. Hasil penelusuran kemudian ditampilkan layar.

Program ini menunjukkan proses penelusuran pada Double Linked List secara maju dan mundur. Dengan adanya pointer next dan prev, data dapat ditelusuri dari dua arah sehingga proses akses data menjadi lebih fleksibel dibandingkan Single Linked List.

d) HapusDLL_2511532007

```

1 package paket_2511532007;
2
3 public class HapusDLL_2511532007 {
4     //fungsi menghapus node awal
5     public static NodeDLL_2511532007 delHead (NodeDLL_2511532007 head_2007) {
6         if (head_2007 == null) {
7             return null;
8         }
9         NodeDLL_2511532007 temp = head_2007;
10        head_2007 = head_2007.next_2007;
11        if (head_2007 != null) {
12            head_2007.prev_2007 = null;
13        }
14        return head_2007;
15    }
16
17    //fungsi menghapus di akhir
18    public static NodeDLL_2511532007 delLast (NodeDLL_2511532007 head_2007) {
19        if (head_2007 == null) {
20            return null;
21        }
22        if (head_2007.next_2007 == null) {
23            return null;
24        }
25        NodeDLL_2511532007 curr_2007 = head_2007;
26        while (curr_2007.next_2007 != null) {
27            curr_2007 = curr_2007.next_2007;
28        }
29        //update pointer previous node
30        if (curr_2007.prev_2007 != null) {
31            curr_2007.prev_2007.next_2007 = null;
32        }
33        return head_2007;
34    }
35
36    //fungsi menghapus node sesuai index
37    public static NodeDLL_2511532007 delPos (NodeDLL_2511532007 head_2007, int pos_2007) {
38        //jika list kosong
39        if (head_2007 == null) {
40            return head_2007;
41        }
42        NodeDLL_2511532007 curr_2007 = head_2007;
43        //jika index yang di hapus sama dengan 0
44        for (int i = 1; curr_2007 != null && i < pos_2007; ++i) {
45            curr_2007 = curr_2007.next_2007;
46        }
47        //jika index tidak ditemukan
48        if (curr_2007 == null) {
49            return head_2007;
50        }
51        //jika index ditemukan
52        if (curr_2007.next_2007 != null) {
53            curr_2007.prev_2007.next_2007 = curr_2007.next_2007;
54        }
55        if (curr_2007.next_2007 != null) {
56            curr_2007.next_2007.prev_2007 = curr_2007.prev_2007;
57        }
58        //jika index dihapus head
59        if (head_2007 == curr_2007) {
60            head_2007 = curr_2007.next_2007;
61        }
62        return head_2007;
63    }
64
65    //fungsi menampilkan list
66    public static void printList (NodeDLL_2511532007 head_2007) {
67        NodeDLL_2511532007 curr_2007 = head_2007;
68        while (curr_2007 != null) {
69            System.out.print(curr_2007.data_2007 + " ");
70            curr_2007 = curr_2007.next_2007;
71        }
72        System.out.println();
73    }
74
75    public static void main(String[] args) {
76        // buat sebuah list
77        NodeDLL_2511532007 head_2007 = new NodeDLL_2511532007(1);
78        head_2007.next_2007 = new NodeDLL_2511532007(2);
79        head_2007.next_2007.prev_2007 = head_2007;
80        head_2007.next_2007.next_2007 = new NodeDLL_2511532007(3);
81        head_2007.next_2007.next_2007.prev_2007 = head_2007;
82        head_2007.next_2007.next_2007.next_2007 = new NodeDLL_2511532007(4);
83        head_2007.next_2007.next_2007.next_2007.prev_2007 = head_2007;
84        head_2007.next_2007.next_2007.next_2007.next_2007 = new NodeDLL_2511532007(5);
85        head_2007.next_2007.next_2007.next_2007.next_2007.prev_2007 = head_2007;
86    }
87
88    //jika index ditemukan
89    if (curr_2007 != null) {
90        curr_2007 = curr_2007.next_2007;
91    }
92    //jika index tidak ditemukan
93    if (curr_2007 == null) {
94        return head_2007;
95    }
96    //jika index ditemukan
97    if (curr_2007.next_2007 != null) {
98        curr_2007.prev_2007.next_2007 = curr_2007.next_2007;
99    }
100   if (curr_2007.next_2007 != null) {
101       curr_2007.next_2007.prev_2007 = curr_2007.prev_2007;
102   }
103   //jika index dihapus head
104   if (head_2007 == curr_2007) {
105       head_2007 = curr_2007.next_2007;
106   }
107   return head_2007;
108 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)
Program diawali dengan package `pekan6_2511532007`; untuk mengelompokkan file Java. Kemudian dibuat public class `HapusDLL_2511532007` sebagai kelas utama yang berisi implementasi penghapusan data pada Double Linked List.
2. Method `delHead()` (menghapus node awal)
Method `delHead()` digunakan untuk menghapus node pertama pada Double Linked List. Jika linked list kosong, maka program akan mengembalikan nilai null. Jika terdapat data, maka head akan dipindahkan ke node berikutnya dan pointer prev pada head baru diatur menjadi null.
3. Method `delLast()` (menghapus node terakhir)
Method `delLast()` digunakan untuk menghapus node terakhir pada Double Linked List. Jika linked list kosong atau hanya memiliki satu node, maka hasilnya menjadi null. Jika terdapat lebih dari satu node, program akan menelusuri hingga node terakhir lalu memutus hubungan node terakhir dengan node sebelumnya.
4. Method `delPos()` (menghapus node pada posisi tertentu)
Method `delPos()` digunakan untuk menghapus node pada posisi tertentu. Program akan menelusuri linked list hingga posisi yang dimaksud. Setelah node ditemukan, pointer node sebelumnya dan node berikutnya akan diperbarui agar node yang dihapus tidak lagi terhubung dalam linked list. Jika posisi tidak ditemukan, maka linked list tidak berubah.
5. Method `printList()` (menampilkan isi linked list)
Method `printList()` digunakan untuk menampilkan seluruh isi Double Linked List dari awal hingga akhir. Program akan menampilkan data satu per satu secara berurutan.

6. Method `main()` (program utama)

Method `main` digunakan untuk menjalankan program. Pertama dibuat Double Linked List awal dengan data 1 2 3 4 5. Setelah itu linked list ditampilkan. Selanjutnya dilakukan beberapa proses penghapusan, yaitu menghapus node pertama, menghapus node terakhir, dan menghapus node pada posisi ke-2. Setiap perubahan langsung ditampilkan untuk melihat hasil linked list terbaru.

7. Alur kerja program

Program dimulai dengan membuat Double Linked List awal. Setelah itu program melakukan beberapa operasi penghapusan node, yaitu di awal, di akhir, dan pada posisi tertentu. Setelah setiap proses penghapusan, isi linked list akan diperbarui dan ditampilkan kembali.

Kesimpulan:

Program ini menunjukkan proses penghapusan node pada Double Linked List di berbagai posisi, yaitu di awal, di akhir, dan di tengah. Dengan adanya pointer `next` dan `prev`, hubungan antar node dapat diperbarui dengan mudah sehingga proses pengelolaan data menjadi lebih fleksibel dan dinamis.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan struktur data yang digunakan untuk menyimpan data secara berurutan dengan dua pointer, yaitu pointer ke node berikutnya dan node sebelumnya. Dengan adanya dua pointer tersebut, proses penambahan, penghapusan, dan penelusuran data dapat dilakukan dari dua arah, yaitu maju dan mundur. Melalui praktikum ini, dapat dipahami cara membuat node, menambahkan data, menghapus data, dan melakukan traversal pada Double Linked List menggunakan bahasa pemrograman Java.

3.2 Saran

Sebaiknya praktikum dilakukan dengan lebih teliti dalam mengatur pointer next dan prev agar hubungan antar node tidak terjadi kesalahan. Selain itu, mahasiswa disarankan untuk lebih sering mencoba berbagai variasi program Double Linked List agar lebih memahami cara kerja struktur data ini. Pemahaman dasar tentang Double Linked List juga perlu ditingkatkan karena materi ini menjadi dasar untuk mempelajari struktur data yang lebih kompleks.

DAFTAR PUSTAKA

- [1] [Telkom University Jakarta](#), “Linked List: Single, Double, dan Circular Linked List,” diakses pada 11 Mei 2026.
- [2] “Modul Praktikum Algoritma & Struktur Data – Linked List,” [Universitas Negeri Malang](#), diakses pada 11 Mei 2026.
- [3] [TutorialsPoint](#), “Doubly Linked List Algorithm,” diakses pada 11 Mei 2026.

TUGAS PRAKTIKUM PEKAN 6

1. Kode Program Lagu_2511532007

```
package pekan6_2511532007;

// class node lagu
public class Lagu_2511532007 {
    String judul_2007;
    String penyanyi_2007;

    Lagu_2511532007 next_2007;
    Lagu_2511532007 prev_2007;

    // constructor
    public Lagu_2511532007(String judul_2007, String
    penyanyi_2007) {
        this.judul_2007 = judul_2007;
        this.penyanyi_2007 = penyanyi_2007;
        this.next_2007 = null;
        this.prev_2007 = null;
    }

    // getter judul
    public String getJudul_2007() {
        return judul_2007;
    }

    // setter judul
    public void setJudul_2007(String judul_2007) {
        this.judul_2007 = judul_2007;
    }

    // getter penyanyi
    public String getPenyanyi_2007() {
        return penyanyi_2007;
    }
}
```

```

    }

    // setter penyanyi
    public void setPenyanyi_2007(String penyanyi_2007) {
        this.penyanyi_2007 = penyanyi_2007;
    }
}

```

2. Kode Program Musik_2511532007

```

package pekan6_2511532007;

import java.util.Scanner;

public class Musik_2511532007 {

    Lagu_2511532007 head_2007 = null;
    Lagu_2511532007 tail_2007 = null;

    // menambah lagu di akhir playlist
    public void tambahLagu_2007(String judul_2007, String
    penyanyi_2007) {

        Lagu_2511532007 laguBaru_2007 =
            new Lagu_2511532007(judul_2007, penyanyi_2007);

        // jika playlist kosong
        if (head_2007 == null) {
            head_2007 = laguBaru_2007;
            tail_2007 = laguBaru_2007;
        } else {
            tail_2007.next_2007 = laguBaru_2007;

```

```

        laguBaru_2007.prev_2007 = tail_2007;
        tail_2007 = laguBaru_2007;
    }

    System.out.println("Lagu berhasil ditambahkan!");
}

// menghapus lagu pertama
public void hapusLaguAwal_2007() {

    // jika playlist kosong
    if (head_2007 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    // jika hanya ada satu lagu
    if (head_2007 == tail_2007) {
        head_2007 = null;
        tail_2007 = null;
    } else {
        head_2007 = head_2007.next_2007;
        head_2007.prev_2007 = null;
    }

    System.out.println("Lagu pertama berhasil dihapus!");
}

// menampilkan playlist dari awal ke akhir
public void tampilMaju_2007() {

    if (head_2007 == null) {

```

```

        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532007 bantu_2007 = head_2007;

    System.out.println("=== Playlist Maju ===");

    while (bantu_2007 != null) {
        System.out.println(
            "Judul : " + bantu_2007.judul_2007 +
            " | Penyanyi : " + bantu_2007.penyanyi_2007);

        bantu_2007 = bantu_2007.next_2007;
    }
}

// menampilkan playlist dari akhir ke awal
public void tampilMundur_2007() {

    if (tail_2007 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532007 bantu_2007 = tail_2007;

    System.out.println("=== Playlist Mundur ===");

    while (bantu_2007 != null) {
        System.out.println(
            "Judul : " + bantu_2007.judul_2007 +

```

```

        " | Penyanyi : " + bantu_2007.penyanyi_2007);

        bantu_2007 = bantu_2007.prev_2007;
    }
}

// mencari lagu berdasarkan judul
public void cariLagu_2007(String judulCari_2007) {

    if (head_2007 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532007 bantu_2007 = head_2007;
    boolean ditemukan_2007 = false;

    while (bantu_2007 != null) {

        if
(bantu_2007.judul_2007.equalsIgnoreCase(judulCari_2007)) {

            System.out.println("Lagu ditemukan!");
            System.out.println(
                "Judul : " + bantu_2007.judul_2007);
            System.out.println(
                "Penyanyi : " + bantu_2007.penyanyi_2007);

            ditemukan_2007 = true;
            break;
        }
    }
}

```

```

        bantu_2007 = bantu_2007.next_2007;
    }

    if (!ditemukan_2007) {
        System.out.println("Lagu tidak ditemukan!");
    }
}

// program utama
public static void main(String[] args) {

    Scanner input_2007 = new Scanner(System.in);

    Musik_2511532007 playlist_2007 =
        new Musik_2511532007();

    int pilihan_2007;

    do {

        System.out.println("\n=== Playlist Musik NIM: 2511532007
        ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");

        System.out.print("Pilihan: ");
        pilihan_2007 = input_2007.nextInt();
        input_2007.nextLine();
    } while (pilihan_2007 != 6);
}

```



```
switch (pilihan_2007) {

    case 1:

        System.out.print("Judul Lagu : ");
        String judul_2007 =
            input_2007.nextLine();

        System.out.print("Penyanyi : ");
        String penyanyi_2007 =
            input_2007.nextLine();

        playlist_2007.tambahLagu_2007(
            judul_2007,
            penyanyi_2007);

        break;

    case 2:

        playlist_2007.hapusLaguAwal_2007();

        break;

    case 3:

        playlist_2007.tampilMaju_2007();

        break;

    case 4:
```

```
        playlist_2007.tampilMundur_2007();

        break;

    case 5:

        System.out.print("Masukkan judul lagu: ");

        String cari_2007 =
            input_2007.nextLine();

        playlist_2007.cariLagu_2007(
            cari_2007);

        break;

    case 6:

        System.out.println("Program selesai.");
        break;

    default:

        System.out.println("Pilihan tidak valid!");
    }

} while (pilihan_2007 != 6);

input_2007.close();
}
}
```

3. Output

```

package-info: package pekant_2511532007;
import java.util.Scanner;
public class Musik_2511532007 {
    Lagu_2511532007 head_2007 = null;
    Lagu_2511532007 tail_2007 = null;
    // menambah lagu di akhir playlist
    public void tambahLagu_2007(String judul_2007, String penyanyi_2007) {
        Lagu_2511532007 laguBaru_2007 =
            new Lagu_2511532007(judul_2007, penyanyi_2007);
        // jika playlist kosong
        if (head_2007 == null) {
            head_2007 = laguBaru_2007;
            tail_2007 = laguBaru_2007;
        } else {
            tail_2007.next_2007 = laguBaru_2007;
            laguBaru_2007.prev_2007 = tail_2007;
            tail_2007 = laguBaru_2007;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }
    // menghapus lagu pertama
    public void hapusLaguawal_2007() {
        // jika playlist kosong
        if (head_2007 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        // jika hanya ada satu lagu
        if (head_2007 == tail_2007) {
            head_2007 = null;
            tail_2007 = null;
        }
    }
}

```

Musik 2511532007 [Java Application] C:\Users\VIP.p2\pool\plugins\org.eclipse.jdt.ui\openjdk.http...

```

--- Playlist Musik NIM: 2511532007 ---
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu : Rio Cristo Pingo Diamante
Penyanyi : Rosalia
Lagu berhasil ditambahkan!

--- Playlist Musik NIM: 2511532007 ---
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Judul : Rio Cristo Pingo Diamante | Penyanyi : Rosalia

--- Playlist Musik NIM: 2511532007 ---
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

```

Jika isi playlist kosong, tidak menghasilkan error pada output

```

package-info: package pekant_2511532007;
import java.util.Scanner;
public class Musik_2511532007 {
    Lagu_2511532007 head_2007 = null;
    Lagu_2511532007 tail_2007 = null;
    // menambah lagu di akhir playlist
    public void tambahLagu_2007(String judul_2007, String penyanyi_2007) {
        Lagu_2511532007 laguBaru_2007 =
            new Lagu_2511532007(judul_2007, penyanyi_2007);
        // jika playlist kosong
        if (head_2007 == null) {
            head_2007 = laguBaru_2007;
            tail_2007 = laguBaru_2007;
        } else {
            tail_2007.next_2007 = laguBaru_2007;
            laguBaru_2007.prev_2007 = tail_2007;
            tail_2007 = laguBaru_2007;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }
    // menghapus lagu pertama
    public void hapusLaguawal_2007() {
        // jika playlist kosong
        if (head_2007 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        // jika hanya ada satu lagu
        if (head_2007 == tail_2007) {
            head_2007 = null;
            tail_2007 = null;
        }
    }
}

```

Musik 2511532007 [Java Application] C:\Users\VIP.p2\pool\plugins\org.eclipse.jdt.ui\openjdk.http...

```

--- Playlist Musik NIM: 2511532007 ---
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist kosong!

--- Playlist Musik NIM: 2511532007 ---
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

```

4. Kesimpulan

Program yang dibuat merupakan implementasi Double Linked List pada pengelolaan playlist lagu sederhana menggunakan bahasa Java. Setiap lagu disimpan dalam bentuk node yang memiliki data judul lagu, penyanyi, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, playlist dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Melalui program ini, pengguna dapat menambahkan lagu ke dalam playlist, menghapus lagu pertama, menampilkan daftar lagu secara maju dan mundur, serta mencari lagu berdasarkan judul. Program juga sudah menangani kondisi ketika playlist kosong sehingga tidak menyebabkan error saat dijalankan.